

Design and Implementation of a ROS 2-Based Autonomous Indoor Mapping and Navigation System for a Low-Cost Differential-Drive Robot

M. A. Abdul Rafi¹

¹Department of Computer Science & Engineering
University of Moratuwa, Sri Lanka
Email: abdulr.23@cse.mrt.ac.lk

Abstract

Reliable indoor autonomous navigation remains a core problem in mobile robotics, requiring the coupled solution of mapping, localization, path planning, and control on constrained embedded hardware. This paper presents the design and implementation of a low-cost, LiDAR-only autonomous navigation system built on a Kobuki differential-drive base, a Raspberry Pi, and a 2D LD19 LiDAR, running under ROS 2 Jazzy. The system uses SLAM Toolbox for real-time pose-graph mapping and localization, a custom eight-connected A planner with obstacle inflation and path smoothing for global planning, and a Pure-Pursuit controller with proportional heading correction for trajectory tracking. A React-based web interface, connected through `rosbridge`, allows an operator to issue click-to-goal commands and manual joystick overrides without direct terminal access. Experimental trials in an indoor office environment show that the robot consistently reaches goals within a 0.35 m tolerance, safely rejects goals located inside mapped obstacles, and smoothly re-plans when a new goal is issued mid-trajectory. The results demonstrate that a fully autonomous, closed-loop navigation stack can be realized on inexpensive, readily available hardware while retaining the modularity of a standard ROS 2 navigation pipeline.*

Index Terms—Autonomous Navigation, SLAM, LiDAR, A* Path Planning, Pure Pursuit, ROS 2, Mobile Robotics

I. INTRODUCTION

Autonomous indoor navigation requires a mobile robot to simultaneously answer four coupled questions: what does the environment look like, where is the robot within that environment, how should it move to reach a destination safely, and how should the wheels be actuated to follow that trajectory. Solving mapping, localization, planning, and control together, in real time and on modest embedded hardware, remains a practically significant challenge, particularly for platforms intended for research and educational use where cost and reproducibility matter as much as raw performance.

Many indoor navigation systems rely on RGB-D cameras or expensive 3D LiDAR units to achieve robust obstacle detection, which increases both hardware cost and on-board computational load. This work instead investigates a LiDAR-only 2D perception pipeline, arguing that a single planar LiDAR, combined with an efficient graph-based planner and a light-weight geometric controller, is sufficient for reliable navigation in structured indoor spaces while remaining tractable on a Raspberry Pi.

This paper describes the design, implementation, and evaluation of such a system, built around a Kobuki robot base, an LD19 2D LiDAR, and the ROS 2 Jazzy middleware. The main contributions of this work are: (1) an integrated ROS 2

navigation stack combining SLAM Toolbox mapping with a custom A* global planner and a Pure-Pursuit local controller; (2) a browser-based human-robot interface, connected via `rosbridge`, that removes the need for terminal-level operation; and (3) an experimental evaluation of the system's path-planning accuracy, goal-rejection safety, and re-planning behaviour in a real indoor environment.

II. LITERATURE REVIEW

Path planning on a discretized map is classically addressed using the A* search algorithm, which combines the actual cost from the start node with a heuristic estimate of the remaining cost to guarantee an optimal path when the heuristic is admissible [1]. A* and its many grid-based variants remain the dominant approach for global planning in occupancy-grid-based mobile robotics because of their predictable optimality guarantees and straightforward implementation.

For trajectory tracking, the Pure-Pursuit algorithm, originally developed at Carnegie Mellon University for the Teragator and NavLab vehicles, computes a curvature that carries the vehicle toward a look-ahead point on the reference path, and has since become a standard geometric controller for wheeled mobile robots due to its simplicity and stability across a wide range of speeds [2].

The representation of the environment as a probabilistic occupancy grid, in which each cell holds the likelihood of being occupied, free, or unknown, was introduced by Elfes as a robust way to fuse noisy range-sensor data into a consistent spatial map [3], and this representation underlies nearly all modern 2D SLAM and navigation stacks.

The Robot Operating System (ROS) provides the middleware conventions, message-passing infrastructure, and tooling that have made modular, reusable navigation software practical across heterogeneous robot platforms [4]. Building on this ecosystem, SLAM Toolbox implements pose-graph-based simultaneous localization and mapping with support for continued mapping, optimization-based localization, and lifelong map maintenance, and is the currently recommended 2D SLAM package for ROS 2 [5]. At the navigation-stack level, Navigation2 (Nav2) reimplements the classical ROS Navigation stack using behaviour trees and lifecycle-managed nodes suited to the real-time and safety requirements of ROS 2, and has been validated in long-duration, pedestrian-populated deployments [6]. The present work adopts the same general architecture, SLAM-based mapping feeding a discrete global planner and a geometric local controller, while implementing a custom, lightweight A* planner and Pure-Pursuit controller tailored to a low-cost Raspberry-Pi-class platform.

III. MATERIALS AND METHODS

A. Hardware Design

The platform is built around a Kobuki differential-drive base, chosen for its widely documented kinematics and existing ROS driver support. Perception is provided by a single LD19 2D LiDAR performing 360° planar sweeps at a serial baud rate of 230,400, mounted centrally on top of the chassis. All onboard computation, including SLAM, planning, and control, runs on a Raspberry Pi flashed with Ubuntu and ROS 2 Jazzy. No RGB-D camera or additional exteroceptive sensor is used; this LiDAR-only design choice keeps perception lighting-independent and keeps CPU utilization low enough to run comfortably on the Raspberry Pi's limited compute budget.

B. Software Architecture

The software is organized as a set of communicating ROS 2 nodes launched from a single hardware bring-up file. Figure 1 (omitted here for brevity) summarizes the flow of data: LiDAR scans on the `/scan` topic feed SLAM Toolbox, which publishes the occupancy-grid map on `/map` together with the `map→base_link` transform; the navigation server consumes the map and the robot's current pose to plan and execute paths; and a custom Kobuki controller node converts velocity commands into low-level motor commands. An operator interacts with the system through a React web application, served locally and connected to the ROS 2 graph via `rosbridge_websocket` over port 9090, which allows click-to-goal navigation and manual joystick teleoperation from any browser on the same network, removing the

need for direct terminal access to the robot.

C. Mapping and Localization

SLAM Toolbox performs pose-graph-based 2D SLAM, consuming raw LiDAR scans and an odometry seed to incrementally build and refine an occupancy grid, where each cell is labelled free, occupied, or unknown. The resulting map and the robot's estimated pose are published continuously and are frozen during autonomous navigation so that cell indices computed during planning remain valid for the duration of a navigation action.

D. Global Path Planning

Global planning uses a custom implementation of the A* search algorithm operating directly on the occupancy grid. The planner performs an eight-connected search, assigning a cost of 1.0 to axis-aligned moves and $\sqrt{2} \approx 1.414$ to diagonal moves, and evaluates each candidate cell using the standard cost function

$$f(n) = g(n) + h(n), \quad (1)$$

where $g(n)$ is the accumulated cost from the start cell and $h(n)$ is a heuristic estimate of the remaining distance to the goal. Prior to search, obstacles in the map are inflated by a configurable safety margin, and any candidate path cell that falls within this inflated region, or within an unmapped (unknown) cell, is rejected during the collision-checking stage. Once a raw grid path is found, it is post-processed with a path-smoothing step that removes redundant vertices and sharp zig-zag segments, producing a shorter and more mechanically feasible trajectory for the controller to track.

E. Trajectory Control

Path tracking is performed with a Pure-Pursuit controller using a fixed look-ahead distance of 0.6 m. At each control cycle, the controller selects the path point at the look-ahead distance ahead of the robot, computes the heading error between the robot's current orientation and the bearing to that point, and applies a proportional correction

$$\omega = K_p e_\theta, \quad (2)$$

where e_θ is the heading error and K_p is a tuned proportional gain. Linear velocity is additionally modulated as a function of local path curvature, so that the robot slows through turns and accelerates on straight segments, improving tracking accuracy without sacrificing average travel speed.

F. System Integration

All nodes are launched together and coordinated through a custom ROS 2 navigation action that connects the A* planner and the Pure-Pursuit controller, using TF2 for consistent coordinate transforms and multi-threaded executors so that sensor processing and control run concurrently. When a new goal is received while a previous navigation action is still executing, the action client cancels the in-progress goal and seamlessly redirects the robot to the new target.

IV. RESULTS AND DISCUSSION

A. Performance Analysis

The system was evaluated through repeated indoor trials in a mapped office-like environment with static obstacles.

- **Goal accuracy:** the robot consistently planned, smoothed, and halted within the configured waypoint tolerance of 0.35 m of the commanded goal.
- **Safety rejection:** goals clicked inside mapped walls or inflated obstacle regions were correctly rejected before execution, with the navigation action aborting safely rather than attempting an infeasible path.
- **Dynamic re-direction:** mid-route goal changes were handled cleanly, with the action client cancelling the active trajectory and computing a new path from the robot's current pose without manual intervention.

B. Challenges and Limitations

Several practical challenges were encountered during development. Default simulation-time settings caused the TF transform tree to break until the hardware clock and frame configuration were explicitly forced in the launch files. The per-cell inflation check, which scans a 7×7 neighbourhood around every expanded A* node, proved computationally heavy relative to the Raspberry Pi's processing budget. In addition, the current design intentionally freezes the occupancy map during navigation to keep planning cell indices consistent, which prevents the system from reacting to obstacles that appear after planning begins; unknown (-1) cells are also not currently rejected outright during collision checking, allowing the planner to route through unmapped regions in some cases.

V. CONCLUSION AND FUTURE WORK

This work presented a low-cost, LiDAR-only autonomous navigation system for a Kobuki-based mobile robot, integrating SLAM Toolbox mapping, a custom A* global planner with obstacle inflation and path smoothing, and a Pure-Pursuit trajectory controller within ROS 2 Jazzy, all exposed through a browser-based operator interface. Trials in an indoor environment confirm that the system reliably reaches goals within tolerance, rejects unsafe goals, and re-plans smoothly in response to new commands, demonstrating that a complete autonomous navigation pipeline can be realized on inexpensive, widely available hardware.

Future work will focus on pre-computing a global costmap so that obstacle inflation is performed once rather than per expanded node, incorporating reactive obstacle avoidance for dynamic, unmapped obstacles such as pedestrians, unfreezing the map to support continuous replanning at higher frequency, and explicitly excluding unknown cells from the planner's set of valid states.

REFERENCES

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [2] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, Jan. 1992.
- [3] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *IEEE Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [4] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: An open-source robot operating system," in *Proc. IEEE ICRA Workshop on Open Source Software*, 2009.
- [5] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [6] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2020.
- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths,"